



AUDIT REPORT

September 2026

For



Zoth

Table of Content

Executive Summary	04
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	09
Techniques and Methods	11
Types of Severity	13
Types of Issues	14
Severity Matrix	15
■ High Severity Issues	16
1. Attacker will extract excess arbitrage value from the vault as the UTC-day limit reset combines with the hardcoded stablecoin rate during a depeg	16
■ Medium Severity Issues	17
1. A vault admin will process an approval for a now-noncompliant party as request-approval never re-runs the compliance gate checked at request creation	17
2. min_first_deposit Can Be Bypassed by Depositing on Behalf of Another User	18
3. Request approval recalculates the payout amount but validates nothing, so a request can settle for zero	20
4. Pause does not cover the approval paths, and a granular pause can silently do nothing	25
■ Low Severity Issues	30

1. zOPAL can be added as a payment token in its own vaults	30
2. The last DEFAULT_ADMIN_ROLE holder can remove itself and permanently lock role management	33
3. Oracle max_staleness has a lower bound but no upper bound, and a new price is usable immediately	36
4. Payment-token decimals are read live from metadata on every call instead of being stored at listing	38
5. initialize skips the sanctions-list validation that set_sanctions_list performs	42
6. The settlement store has no obligation tracking, and request_redeemer is never enforced	45
7. Economic setters accept any value with no bounds or coherence checks	48
8. Removing a payment token strands every pending request denominated in it	51
9. A rejected deposit request never returns the token allowance it consumed	53
■ Informational Severity Issues	57
1. zopal_token stores a TransferRef and an ExtendRef that no code ever uses	57
2. friend declarations grant more access than any caller actually uses	61
Functional Tests	63
Automated Tests	65
Closing Summary & Disclaimer	66

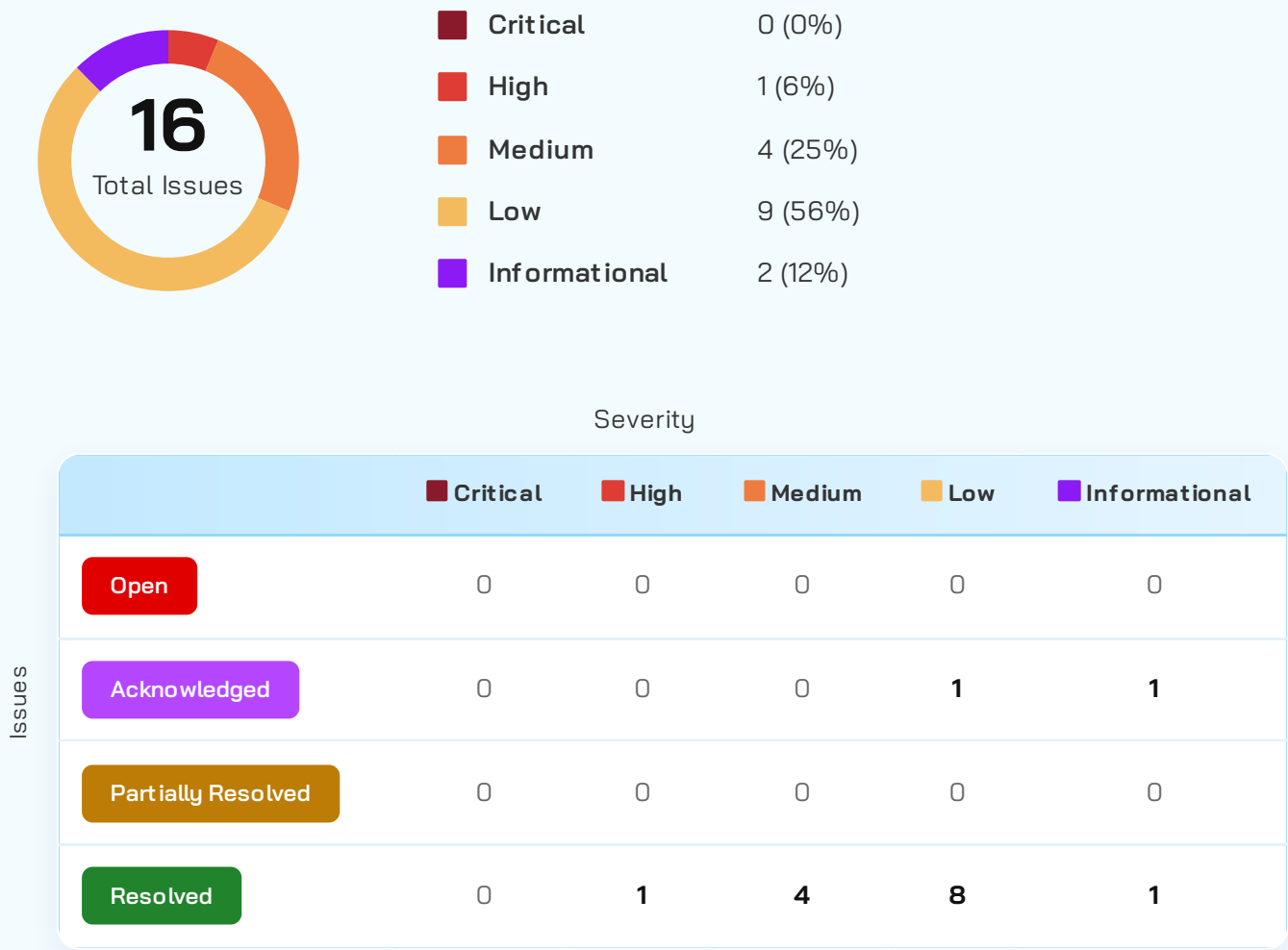
Executive Summary

Project Name	Zoth
Protocol Type	Vault
Project URL	https://www.zoth.io/
Overview	zVault is a tokenized fund on Movement. Users deposit a stablecoin (USDCx) into the deposit vault and receive zOPAL, a share token minted at the fund's current net asset value as published by an on-chain oracle. To exit, users go to the redemption vault, which offers three routes: an instant redemption paid straight from a liquidity buffer, a redemption request that an admin approves and settles from a separate settlement store, and a fiat request that burns the shares against an off-chain bank transfer.
Audit Scope	The scope of this Audit was to analyze the ZVault Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/MoveIndustries/zvault-movement
Contracts in Scope	access_control.move, compliance.move, deposit_vault.move, redemption_vault.move, vault_base.move, zopal_token.move, oracle_access_control.move, price_oracle.move
Branch	main
Commit Hash	2fb43f00699853446761000c539bbc013249e19c
Language	Move
Blockchain	Movement Chain
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	6th August 2026 - 17th August 2026

Updated Code Received	4th September 2026
Review 2	4th September 2026
Fixed In	7f607489b127fd4ade3b1c95bd8da2fc9091834f

✔ Verify the Authenticity of Report on QuillAudits Leaderboard:
<https://www.quillaudits.com/leaderboard>

Number of Issues per Severity



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Attacker will extract excess arbitrage value from the vault as the UTC-day limit reset combines with the hardcoded stablecoin rate during a depeg	High	Resolved
2	A vault admin will process an approval for a now-noncompliant party as request-approval never re-runs the compliance gate checked at request creation	Medium	Resolved
3	min_first_deposit Can Be Bypassed by Depositing on Behalf of Another User	Medium	Resolved
4	Request approval recalculates the payout amount but validates nothing, so a request can settle for zero	Medium	Resolved
5	Pause does not cover the approval paths, and a granular pause can silently do nothing	Medium	Resolved
6	zOPAL can be added as a payment token in its own vaults	Low	Resolved
7	The last DEFAULT_ADMIN_ROLE holder can remove itself and permanently lock role management	Low	Resolved

Issue No.	Issue Title	Severity	Status
8	Oracle max_staleness has a lower bound but no upper bound, and a new price is usable immediately	Low	Resolved
9	Payment-token decimals are read live from metadata on every call instead of being stored at listing	Low	Resolved
10	initialize skips the sanctions-list validation that set_sanctions_list performs	Low	Resolved
11	The settlement store has no obligation tracking, and request_redeemer is never enforced	Low	Resolved
12	Economic setters accept any value with no bounds or coherence checks	Low	Resolved
13	Removing a payment token strands every pending request denominated in it	Low	Resolved
14	A rejected deposit request never returns the token allowance it consumed	Low	Acknowledged
15	zopaL_token stores a TransferRef and an ExtendRef that no code ever uses	Informational	Resolved
16	friend declarations grant more access than any caller actually uses	Informational	Acknowledged

Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls

✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Style guide violation

✓ Using throw

✓ Unsafe type inference

✓ Using inline assembly

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open Security vulnerabilities identified that must be resolved and are currently unresolved.	Resolved These are the issues identified in the initial audit and have been successfully fixed.
Acknowledged Vulnerabilities which have been acknowledged but are yet to be resolved.	Partially Resolved Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		 High	 Medium	 Low
Likelihood	 High	Critical	High	Medium
	 Medium	High	Medium	Low
	 Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

Attacker will extract excess arbitrage value from the vault as the UTC-day limit reset combines with the hardcoded stablecoin rate during a depeg

Resolved

Path

```
move/zvault_core/sources/vault_base.move
```

Function Name

```
require_and_update_limit, get_token_rate
```

Description

The use of a fixed UTC-calendar-day bucket for the daily volume throttle will cause a doubling of the intended daily exposure limit for the vault as an attacker will time two transactions across a UTC day boundary (e.g. 23:59:59 and 00:00:01) to reset the accumulator early. Combined with `get_token_rate` returning a hardcoded `STABLECOIN_RATE_8` instead of a live price, this will cause an excess arbitrage loss for the protocol/LPs as an attacker will exploit both the doubled volume window and a depegged "stable" payment token simultaneously to extract roughly double the value that either flaw alone would allow before the token is repriced or delisted.

In `vault_base.move#L336-L342`, `require_and_update_limit` computes `day = timestamp::now_seconds() / SECONDS_PER_DAY`, so the daily volume accumulator resets the instant the wall clock crosses a UTC day boundary, this is more like a hard cliff rather than enforcing a rolling 24-hour window.

In `vault_base.move:L581-587`, `get_token_rate` unconditionally returns the compile-time constant `STABLECOIN_RATE_8` for every admitted payment token marked `stable == true`, instead of reading any live or per-token price input, so it never reflects a token that has depegged from \$1.00.

Impact

High: Attacker will extract excess arbitrage value from the vault as the UTC-day limit reset combines with the hardcoded stablecoin rate during a depeg.

Likelihood

Medium, requires depeg event.

Recommendation

Replace the hardcoded `STABLECOIN_RATE_8` with a live oracle-fed price per token (with staleness and deviation checks) and consider replacing the fixed UTC-calendar-day bucket with a rolling 24-hour window.

Medium Severity Issues

A vault admin will process an approval for a now-noncompliant party as request-approval never re-runs the compliance gate checked at request creation

Resolved

Path

```
move/zvault_core/sources/deposit_vault.move,  
move/zvault_core/sources/redemption_vault.move
```

Function Name

```
approve_request_internal (both vaults), safe_bulk_approve_request_with_rate
```

Description

approve_request_internal on both vaults never calls any compliance function, so a request created by a compliant user is approved unconditionally regardless of any blacklist or sanctions status change that occurred while the request was pending.

Impact

This is a compliance and regulatory-exposure impact rather than a direct extraction of protocol value, and is fully avoidable with a manual re-check before approval.

Likelihood

High as no external preconditions are required. Only internal preconditions are: 1. A request exists in pending status. 2. The request's sender or recipient becomes non-compliant (blacklisted or sanctioned) after the request.

Recommendation

re-runs compliance::assert_user_access on pending requests. To be called in approve_request_internal on both vaults.

Medium Severity Issues

min_first_deposit Can Be Bypassed by Depositing on Behalf of Another User

Resolved

Path

```
move/zvault_core/sources/deposit_vault.move
```

Function Name

```
calc_and_validate_deposit
```

Description

The min_first_deposit requirement can be bypassed when a user deposits funds on behalf of another account.

The first-deposit check in deposit_vault::calc_and_validate_deposit determines whether the depositor has made a previous deposit by checking state.total_minted using sender.

However, sender represents the caller of the transaction, not necessarily the account receiving the deposited funds.

As a result, an existing user can make a deposit below min_first_deposit on behalf of a new user. This creates a non-zero balance for the recipient without requiring the recipient to satisfy the first-deposit threshold. Once the recipient has received this initial dust-sized deposit, subsequent deposits made directly by the recipient are no longer treated as first deposits.

This effectively allows the protocol's min_first_deposit anti-spam/anti-dust protection to be bypassed.

The protocol has two minimum-deposit protections: min_amount, the general minimum deposit amount, and min_first_deposit, a higher minimum intended to prevent accounts from being initialized with very small/dust deposits.

The problem is that the first-deposit condition is determined using state.total_minted(sender), rather than the account for which the deposit is actually being made. Since the deposit function supports depositing on behalf of another user, the caller and beneficiary can therefore be different accounts.

min_first_deposit Can Be Bypassed by Depositing on Behalf of Another User *(continued)*

```
1 // deposit_vault.move calc_and_validate_deposit
2 if (!vault_base::is_free_from_min(&state.base, sender)) {
3     assert!(mint8 >= (vault_base::min_amount(&state.base) as u128), E_MINT_AMOUNT_LT_MIN);
4     let minted_before =
5         *smart_table::borrow_with_default(&state.total_minted, sender, &0u128);
6     if (minted_before == 0) {
7         assert!(mint8 >= (state.min_first_deposit as u128), E_FIRST_DEPOSIT_LT_MIN);
8     };
9 };
```

Impact

Deposits made on behalf of another account can bypass min_first_deposit requirement / circumvention of the protocol's spam/dust mitigation mechanism.

Likelihood

High as the only precondition is that min_deposit != min_first_deposit.

Recommendation

The first-deposit requirement should be evaluated against the actual beneficiary/recipient account whose balance is being initialized, rather than the transaction caller.

Medium Severity Issues

Request approval recalculates the payout amount but validates nothing, so a request can settle for zero

Resolved

Path

```
move/zvault_core/sources/deposit_vault.move,  
move/zvault_core/sources/redemption_vault.move,  
move/zvault_core/sources/vault_base.move
```

Function Name

```
approve_request_internal (both vaults), redeem_request,  
set_variation_tolerance
```

Description

Requests are validated when they are created and then validated again nowhere. The approval path recomputes the final amount from a fresh admin-supplied rate and applies none of the creation-time checks.

Deposit side: `calc_and_validate_deposit` does the full set of checks at request time. `do_deposit_request` then throws that validated `mint8` away and stores only `usd_amount_without_fees`. The user's tokens leave their wallet at this point. Approval recomputes from scratch. There is no `mint8 > 0` check, no `min_amount` check and no rate sanity check. This is floor division with `new_out_rate` as the divisor, so a large enough rate floors `mint8` to 0. The supply cap check is trivially satisfied by 0, `mint_to` mints nothing, and the request is marked `STATUS_PROCESSED`. The depositor's money is already gone.

Redemption side, plus a bad ordering: the escrowed `zOPAL` is burned at the top of the function, before the payout is even computed. Here the rate is a multiplicand, not a divisor, so a zero or very small rate does not abort; it floors the payout to zero. `approve_request` calls in with `is_safe = false`, which skips the tolerance check entirely. There is no `out_native > 0` guard on either redemption path; the only zero check is `assert!(amount_usd > 0)` on the gross input.

The "safe" path is not a backstop at the boundary: `require_variation_tolerance` compares the deviation against `variation_tolerance_bps`. With `new_price = 0`, `deviation_bps` is exactly 10000. `set_variation_tolerance` calls `validate_fee(tolerance_bps, true)`, which allows `0 < tolerance_bps <= 10000`, so at the reachable setting of 10000 the safe path accepts a zero rate too.

The request path has no slippage protection at all: the instant path takes a bound; the request path does not have the parameter. So a request redeemer has no on-chain protection of any kind against the rate that is applied later.

Request approval recalculates the payout amount but validates nothing, so a request can settle for zero *(continued)*

```
1 // deposit_vault.move calc_and_validate_deposit
2 // Min checks on the POST-FEE mint amount (EVM `_validateMinAmount`).
3 if (!vault_base::is_free_from_min(&state.base, sender)) {
4     assert!(mint8 >= (vault_base::min_amount(&state.base) as u128), E_MINT_AMOUNT_LT_MIN);
5     let minted_before =
6         *smart_table::borrow_with_default(&state.total_minted, sender, &0u128);
7     if (minted_before == 0) {
8         assert!(mint8 >= (state.min_first_deposit as u128), E_FIRST_DEPOSIT_LT_MIN);
9     };
10 };
11 assert!(mint8 > 0, E_INVALID_MINT_AMOUNT);
```

```
1 // deposit_vault.move approve_request_internal
2 // amountZToken = usdAmountWithoutFees * 1e8 / newOutRate (floor).
3 let mint8 = vault_base::mul_div(
4     request.usd_amount_without_fees, vault_base::stablecoin_rate_8(), new_out_rate,
5 );
6
7 let exceeds_cap = zopal_token::total_supply() + mint8 > state.max_supply_cap;
8 if (exceeds_cap) {
9     if (revert_above_cap) { abort E_MAX_SUPPLY_CAP };
10     return false
11 };
12
13 let minted = vault_base::to_u64(mint8);
14 zopal_token::mint_to(request.sender, minted);
```

Request approval recalculates the payout amount but validates nothing, so a request can settle for zero *(continued)*

```
1 // redemption_vault.move approve_request_internal
2 if (is_safe) {
3     vault_base::require_variation_tolerance(&state.base, request.z_token_rate, new_z_token_rate);
4 };
5
6 // Burn the escrowed zOPAL held by the vault identity.
7 zopal_token::burn_from(vault_base::redemption_vault_identity(), request.amount_z_token);
8 release_escrow(state, request.amount_z_token);
9 ...
10 let out_native = vault_base::to_u64(vault_base::mul_div(
11     amount8, new_z_token_rate, request.token_out_rate * boundary,
12 ));
```

```
1 // vault_base.move require_variation_tolerance
2 let diff = if (new_price >= prev_price) { new_price - prev_price } else { prev_price - new_price };
3 let deviation_bps = mul_div(diff, (ONE_HUNDRED_PERCENT as u128), prev_price);
4 assert!(deviation_bps <= (state.variation_tolerance_bps as u128), E_EXCEED_PRICE_DEVIATION);
```

Request approval recalculates the payout amount but validates nothing, so a request can settle for zero *(continued)*

```
1 // redemption_vault.move instant path has a slippage bound
2 public entry fun redeem_instant(
3     user: &signer,
4     token_out: address,
5     amount_z_token_in: u64,
6     min_receive_amount: u64,
7 ) acquires RedemptionVaultState, Identity {
```

```
1 // redemption_vault.move request path has no equivalent parameter
2 public entry fun redeem_request(
3     user: &signer,
4     token_out: address,
5     amount_z_token_in: u64,
6 ) acquires RedemptionVaultState {
```

Impact

Medium. A user can lose the entire value of a request, the deposited tokens on the deposit side, or the escrowed zOPAL on the redemption side, while the request is marked STATUS_PROCESSED, which is indistinguishable on-chain from a successful settlement. Any monitoring that keys on request status sees nothing wrong. There is no recovery path once the status flips.

Request approval recalculates the payout amount but validates nothing, so a request can settle for zero *(continued)***Likelihood**

Medium. The zero-payout case does not need a malicious admin, a decimal or unit mix-up in a rate is enough, and the deposit side aborts on a zero rate (division) while the redemption side silently pays zero, so an operator has no reason to expect the asymmetry. The truncation case can happen at an honest rate when the payout token's decimals round a small escrow amount down to zero.

Recommendation

Guard the rate, re-check the minimums against the freshly computed amount, add a non-zero output assert on both sides.

Medium Severity Issues

Pause does not cover the approval paths, and a granular pause can silently do nothing

Resolved

Path

```
move/zvault_core/sources/vault_base.move,  
move/zvault_core/sources/deposit_vault.move,  
move/zvault_core/sources/redemption_vault.move
```

Function Name

```
pause_fn, assert_fn_not_paused, approve_request_internal (both vaults),  
do_deposit_instant, do_deposit_request
```

Description

Pause is the protocol's emergency stop. It has three gaps, and all three point the same way: the operator believes the vault is stopped when it is not.

1. Only the user-facing entry points are gated. A search for `assert_not_paused` / `assert_fn_not_paused` across both vaults returns five call sites, all of them instant or request entry points. `approve_request_internal` in both vaults starts straight at the request lookup, with no pause check at all. So a paused deposit vault still mints, and a paused redemption vault still burns escrow and pays out. This also contradicts the reason `zopal_token` gives for shipping no token-level pause, that a vault pause already halts mint and burn. It does not.

2. Pausing one key leaves its twin open. There are four separate pause keys on the deposit side. Pausing `deposit_instant` leaves `deposit_instant_with_custom_recipient` fully live, and with `recipient == sender` the two do exactly the same thing. Only two of the four keys have public accessors (`fn_key_deposit_instant`, `fn_key_deposit_request`), so an operator reading the module is pointed at precisely the half that can be bypassed.

3. `pause_fn` accepts any bytes at all. The only assertion is "not already paused", which is trivially true for a key that was never used: `b"depositInstant"` or `b"deposit_instant "` with a trailing space both succeed - the key is stored, a `PauseFn` event fires, monitoring shows a pause, and `deposit_instant` keeps running. `unpause_fn` does validate implicitly, it aborts with `E_FN_NOT_PAUSED` on an unknown key, so the asymmetry looks unintentional.

Pause does not cover the approval paths, and a granular pause can silently do nothing *(continued)*



```

1 redemption_vault: approve_request      safe_approve_request  reject_request
2                   fund_buffer          fund_settlement
3                   withdraw_settlement  withdraw_token
4 deposit_vault:    safe_approve_request  approve_request
5                   safe_bulk_approve_request
6                   safe_bulk_approve_request_with_rate

```



```

1 // deposit_vault.move approve_request_internal: no pause check anywhere in this function
2 fun approve_request_internal(
3   request_id: u64, new_out_rate: u128, is_safe: bool, revert_above_cap: bool,
4 ): bool acquires DepositVaultState {
5   let state = borrow_state_mut();
6   assert!(smart_table::contains(&state.requests, request_id), E_REQUEST_NOT_EXIST);
7   let request = *smart_table::borrow(&state.requests, request_id);
8   assert!(request.status == STATUS_PENDING, E_REQUEST_NOT_PENDING);

```



```

1 // deposit_vault.move per-function pause keys
2 const FN_DEPOSIT_INSTANT: vector<u8> = b"deposit_instant";
3 const FN_DEPOSIT_INSTANT_CUSTOM_RECIPIENT: vector<u8> = b"deposit_instant_custom_recipient";
4 const FN_DEPOSIT_REQUEST: vector<u8> = b"deposit_request";
5 const FN_DEPOSIT_REQUEST_CUSTOM_RECIPIENT: vector<u8> = b"deposit_request_custom_recipient";

```

Pause does not cover the approval paths, and a granular pause can silently do nothing *(continued)*

```
1 // vault_base.move pause_fn
2 public(friend) fun pause_fn(state: &mut VaultState, caller: address, fn_name: vector<u8>) {
3     assert!(
4         !*smart_table::borrow_with_default(&state.fn_paused, fn_name, &false),
5         E_FN_PAUSED,
6     );
7     smart_table::upsert(&mut state.fn_paused, fn_name, true);
8     event::emit(PauseFn { vault: state.vault_identity, caller, fn_name });
9 }
```

```
1 // vault_base.move an unset key reads as UNPAUSED
2 public(friend) fun assert_fn_not_paused(state: &VaultState, fn_name: vector<u8>) {
3     assert_not_paused(state);
4     assert!(
5         !*smart_table::borrow_with_default(&state.fn_paused, fn_name, &false),
6         E_FN_PAUSED,
7     );
8 }
```

Pause does not cover the approval paths, and a granular pause can silently do nothing *(continued)*

```
1 // deposit_vault.move and redemption_vault.move approve_request_internal
2 let state = borrow_state_mut();
3 vault_base::assert_not_paused(&state.base);
4 assert!(smart_table::contains(&state.requests, request_id), E_REQUEST_NOT_EXIST);
```

```
1 // deposit_vault.move do_deposit_instant: the base key is always checked as well
2 {
3     let state = borrow_state();
4     vault_base::assert_fn_not_paused(&state.base, FN_DEPOSIT_INSTANT);
5     if (fn_key != FN_DEPOSIT_INSTANT) {
6         vault_base::assert_fn_not_paused(&state.base, fn_key);
7     };
8     ...
9 }
```

Pause does not cover the approval paths, and a granular pause can silently do nothing *(continued)*

```
1 // vault_base.move pause_fn rejects a key the vault does not know about
2 public(friend) fun pause_fn(state: &mut VaultState, caller: address, fn_name: vector<u8>) {
3     assert!(vector::contains(&state.known_fn_keys, &fn_name), E_UNKNOWN_FN_KEY);
4     assert!(
5         !*smart_table::borrow_with_default(&state.fn_paused, fn_name, &false),
6         E_FN_PAUSED,
7     );
8     smart_table::upsert(&mut state.fn_paused, fn_name, true);
9     event::emit(PauseFn { vault: state.vault_identity, caller, fn_name });
10 }
```

Impact

Medium. During an incident, pausing both vaults still lets the queued approval pipeline mint zOPAL and ship settlement funds, and a granular pause can report success while doing nothing. This is the worst failure mode for an emergency control because it actively tells the operator the wrong thing.

Likelihood

Medium. The harm only shows up during an incident, but that is exactly when the control is guaranteed to be used, and any runbook built on pause is silently wrong. A typo or wrong byte encoding under time pressure is easy, the key constants are module-private, so there is no on-chain way to fetch the exact bytes.

Recommendation

Gate the admin paths, always check the base key, and reject unknown pause keys. `known_fn_keys` is populated in `new_vault_state` from each vault's key constants. Also expose a `#` [view] accessor for all four deposit keys and both redemption keys so operators can read the exact bytes on-chain instead of hand-encoding them. Leave `unpause` and genuine recovery paths exempt from the pause check.

Low Severity Issues

zOPAL can be added as a payment token in its own vaults

Resolved

Path

```
move/zvault_core/sources/vault_base.move,  
move/zvault_core/sources/deposit_vault.move,  
move/zvault_core/sources/redemption_vault.move
```

Function Name

```
add_payment_token, redeem_instant, do_deposit_instant
```

Description

Neither vault stops the share token from being listed as one of its own payment tokens. zOPAL is exactly 8 decimals and exposes a public `metadata_address()`, so it satisfies every guard. `get_token_rate` then prices it at a flat \$1.00 with no oracle involved.

Once it is listed, two things break for any ordinary user.

Draining the redemption buffer: the instant redemption buffer and the pending-request escrow are the same store, `redemption_vault_identity()`. The buffer check counts escrowed zOPAL as available liquidity, and this path never decrements `total_escrowed_z_token`. With NAV at 1.05 and `token_out = zOPAL`, `boundary = pow10(8-8) = 1`, so redeeming 100 zOPAL pays out 105 zOPAL from the same store. That is +5% per round, repeatable until the store is empty, after which every pending approval aborts in `burn_from` and the request queue is stuck.

Minting against itself: on the deposit side, at NAV 0.90, depositing 100 zOPAL mints `mul_div(100e8, 1e8, 0.90e8) = 111.11` zOPAL - 100 surrendered, 111.11 created, backed by nothing.

```
1 // vault_base.move add_payment_token  
2 assert(!vector::contains(&state.payment_tokens, &token), E_TOKEN_ALREADY_ADDED);  
3 assert!(token_decimals <= INTERNAL_DECIMALS, E_UNSUPPORTED_DECIMALS);  
4 assert!(stable, E_NON_STABLE_UNSUPPORTED);  
5 validate_fee(fee_bps, false);
```

zOPAL can be added as a payment token in its own vaults *(continued)*

```
1 // redemption_vault.move redeem_instant buffer check
2 let vault_identity = vault_base::redemption_vault_identity();
3 let out_metadata = token_metadata(token_out);
4 assert!(
5     primary_fungible_store::balance(vault_identity, out_metadata) >= out_native,
6     E_INSUFFICIENT_BUFFER,
7 );
```

```
1 // deposit_vault.move and redemption_vault.move add_payment_token
2 public entry fun add_payment_token(
3     admin: &signer, token: address, fee_bps: u64, allowance: u128, stable: bool,
4 ) acquires RedemptionVaultState {
5     assert_vault_admin(admin);
6     assert!(token != zopal_token::metadata_address(), E_INVALID_TOKEN);
7     let token_decimals = fungible_asset::decimals(token_metadata(token));
8     ...
9 }
```

zOPAL can be added as a payment token in its own vaults *(continued)*

```
1 // redemption_vault.move redeem_instant: escrow is not payable liquidity
2 let available = primary_fungible_store::balance(vault_identity, out_metadata);
3 let reserved = if (token_out == zopal_token::metadata_address()) {
4     state.total_escrowed_z_token
5 } else { 0 };
6 assert!(available >= (out_native as u128) + reserved, E_INSUFFICIENT_BUFFER);
```

Impact

Low. The consequences are severe once the token is listed, but listing it is a configuration action taken by the trusted admin role, and nothing else reaches this state.

Likelihood

Low. Requires the vault admin to call `add_payment_token(zOPAL, ...)`. The concern is that nothing in the code signals that this particular token is dangerous, the guard set gives no hint, and after that both legs are executable by any unprivileged user.

Recommendation

Reject the share token at listing in both vaults, and stop counting escrow as payable buffer liquidity.

Low Severity Issues

The last DEFAULT_ADMIN_ROLE holder can remove itself and permanently lock role management

Resolved

Path

```
move/zvault_core/sources/access_control.move,  
move/zvault_oracle/sources/oracle_access_control.move
```

Function Name

```
revoke_role, renounce_role, get_role_admin
```

Description

Neither role registry has a "cannot remove the last admin" guard, and DEFAULT_ADMIN_ROLE administers itself. `get_role_admin` falls back to DEFAULT_ADMIN_ROLE for any role that has no explicit admin, so the sole holder calling `revoke_role(self, DEFAULT_ADMIN_ROLE, self)` passes `assert_role` and removes itself. `zvault_core::access_control` deliberately omits `renounce_role`, but `revoke_role` reopens the same lockout, which makes that omission cosmetic.

The oracle registry is more direct, it ships `renounce_role` with no last-holder check. `role_admins` is written only in `init_module` and can never be changed afterwards, so the hierarchy cannot be repaired from inside the contract.

```
1 // access_control.move revoke_role has no self-targeting restriction  
2 public entry fun revoke_role(caller: &signer, role: vector<u8>, account: address) acquires Roles {  
3     let sender = signer::address_of(caller);  
4     assert_role(get_role_admin(role), sender);  
5     revoke_internal(role, account, sender);  
6 }
```

The last DEFAULT_ADMIN_ROLE holder can remove itself and permanently lock role management *(continued)*

```
1 // oracle_access_control.move  renounce_role
2 public entry fun renounce_role(caller: &signer, role: vector<u8>, account: address) acquires Roles {
3     let sender = signer::address_of(caller);
4     assert!(sender == account, E_NOT_SELF);
5     revoke_internal(role, account, sender);
6 }
```

```
1 // access_control.move  inside Roles
2 struct Roles has key {
3     members: SmartTable<RoleKey, bool>,
4     role_admins: SmartTable<vector<u8>, vector<u8>>,
5     default_admin_count: u64,
6 }
```

The last DEFAULT_ADMIN_ROLE holder can remove itself and permanently lock role management *(continued)*

```
1 // access_control.move revoke_role
2 public entry fun revoke_role(caller: &signer, role: vector<u8>, account: address) acquires Roles {
3     let sender = signer::address_of(caller);
4     assert_role(get_role_admin(role), sender);
5     if (role == DEFAULT_ADMIN_ROLE && has_role(role, account)) {
6         let roles = borrow_global<Roles>(@zvault_core);
7         assert!(roles.default_admin_count > 1, E_LAST_DEFAULT_ADMIN);
8     };
9     revoke_internal(role, account, sender);
10 }
```

Impact

Low. Serious if it happens, but it can only be reached by the trusted admin acting on itself. On the oracle side PRICE_ADMIN_ROLE and CONFIG_ROLE can never be re-granted, so once the last pushed NAV exceeds max_staleness, get_data_in_base8() aborts and every deposit and redemption reverts permanently, and zvault_oracle is published immutable, so there is no in-place fix, only a redeploy and re-pointing of every consumer. On the core side, vault-admin and operator roles become permanently un-grantable and un-revokable.

Likelihood

Low. Requires an operational key-management mistake by the trusted admin. Worth noting that the two registries have different semantics, one forbids renounce, the other allows it, which is itself an invitation to error.

Recommendation

Track the number of DEFAULT_ADMIN_ROLE holders and refuse to drop below one, in both registries. Apply the identical guard to revoke_role and renounce_role in oracle_access_control.move, and keep default_admin_count in step inside grant_internal / revoke_internal. Fix the oracle side first, that package cannot be patched after deploy. A two-step accept/claim handover for DEFAULT_ADMIN_ROLE is also worth adding.

Low Severity Issues

Oracle max_staleness has a lower bound but no upper bound, and a new price is usable immediately

Resolved

Path

`move/zvault_oracle/sources/price_oracle.move`

Function Name

`set_max_staleness, set_price, get_data_in_base8`

Description

set_max_staleness rejects 0 and nothing else. age is a u64 in seconds, so any sufficiently large value makes the staleness assert in get_data_in_base8 unfalsifiable just as completely as 0 would. This does not need malice, 259200000 is "72 hours" written in milliseconds, an ordinary unit slip, and it is accepted as roughly 3,000 days. The module comment claims the opposite invariant, that the guard is always enforced.

Two related weaknesses sit on the same surface:

No cooldown between a price update and its use. Nothing separates a new NAV from being used to price a live deposit or redemption in the same block, and the deposit path can be configured at 0 bps, so any NAV drift inside the staleness window is capturable at no cost.

The deviation anchor can be walked. within_tolerance compares each update against the immediately preceding value, so the price admin can move the price arbitrarily far in individually compliant steps. set_price skips the check entirely when tolerance_percent == 0, which the setter permits.

```
1 // price_oracle.move set_max_staleness
2 public entry fun set_max_staleness(caller: &signer, new_max_staleness: u64) acquires OracleState {
3     assert_config_role(caller);
4     // Reject 0. Tolerance=0 disabling the deviation check is sanctioned by
5     // audit 4.10; disabling the STALENESS guard is sanctioned nowhere and is
6     // strictly more dangerous (a stuck feed would let vault ops price on
7     // arbitrarily old NAV exactly the audit-4.7 risk the guard prevents).
8     assert!(new_max_staleness > 0, E_ZERO_MAX_STALENESS);
9     ...
10 }
```

Oracle max_staleness has a lower bound but no upper bound, and a new price is usable immediately *(continued)*

```
1 // price_oracle.move
2 const MAX_ALLOWED_STALENESS: u64 = 604800; // 7 days, in seconds
3
4 public entry fun set_max_staleness(caller: &signer, new_max_staleness: u64) acquires OracleState {
5     assert_config_role(caller);
6     assert!(new_max_staleness > 0, E_ZERO_MAX_STALENESS);
7     assert!(new_max_staleness <= MAX_ALLOWED_STALENESS, E_STALENESS_TOO_LARGE);
8     ...
9 }
```

Impact

Low. If the guard is neutralized the NAV can be frozen while the true NAV accrues, and the difference is extractable by any user, for example NAV pinned at 1.00 while true NAV reaches 1.0164 after 60 days at 10% APY, giving roughly \$154/day against a 1e10 daily limit at 0 bps in and 10 bps out, paid for by existing holders. But reaching that state requires the trusted CONFIG_ROLE to set the value. The important part is that zvault_oracle is published immutable, so this cannot be fixed after deploy.

Likelihood

Low. Needs CONFIG_ROLE to pass a wrong value. The free-window capture sub-case needs no privilege and is live at the intended default configuration, but its per-cycle size is small.

Recommendation

Add an upper bound. This has to be done before deploy. Separately: add a floor on tolerance_percent so the deviation check cannot be turned off, consider anchoring the deviation check to a longer window rather than only the previous value, and reconsider the 0 bps deposit fee, which is what makes window capture free rather than merely cheap.

Low Severity Issues

Payment-token decimals are read live from metadata on every call instead of being stored at listing

Resolved

Path

```
move/zvault_core/sources/vault_base.move,  
move/zvault_core/sources/deposit_vault.move,  
move/zvault_core/sources/redemption_vault.move
```

Function Name

```
add_payment_token, calc_and_validate_deposit, redeem_instant,  
approve_request_internal
```

Description

add_payment_token validates that a token has 8 or fewer decimals, but the validated value is not kept anywhere. Every later site therefore re-reads it live from the token's own metadata - the same live read appears in redeem_instant, in the redemption approve_request_internal, and in both add_payment_token wrappers, five places in total.

Balances are raw u64; decimals are only an interpretation of them. An issuer calling fungible_asset::mutate_metadata changes no balance but rescales every valuation the vault does afterwards. The ≤ 8 check is made once at listing time and never maintained.

```
1 // vault_base.move PayTokenConfig has no decimals field  
2 struct PayTokenConfig has store, drop, copy {  
3     fee_bps: u64,  
4     /// Remaining operation allowance, in internal base-8 units.  
5     /// `MAX_ALLOWANCE` means unlimited (not decremented).  
6     allowance: u128,  
7     stable: bool,  
8 }
```

Payment-token decimals are read live from metadata on every call instead of being stored at listing *(continued)*

```
1 // deposit_vault.move  calc_and_validate_deposit
2 // Native decimals come from the FA metadata at call time (audit 4.4:
3 // native units at the boundary, high-precision internally).
4 let token_decimals = fungible_asset::decimals(token_metadata(token_in));
```

```
1 // vault_base.move
2 struct PayTokenConfig has store, drop, copy {
3     fee_bps: u64,
4     allowance: u128,
5     stable: bool,
6     decimals: u8,
7 }
```

Payment-token decimals are read live from metadata on every call instead of being stored at listing *(continued)*

```
1 // vault_base.move add_payment_token
2 assert!(token_decimals <= INTERNAL_DECIMALS, E_UNSUPPORTED_DECIMALS);
3 assert!(stable, E_NON_STABLE_UNSUPPORTED);
4 validate_fee(fee_bps, false);
5 vector::push_back(&mut state.payment_tokens, token);
6 smart_table::upsert(
7     &mut state.token_configs,
8     token,
9     PayTokenConfig { fee_bps, allowance, stable, decimals: token_decimals },
10 );
```

```
1 // vault_base.move new accessor
2 public(friend) fun token_decimals(state: &VaultState, token: address): u8 {
3     assert_token_exists(state, token);
4     smart_table::borrow(&state.token_configs, token).decimals
5 }
```

Payment-token decimals are read live from metadata on every call instead of being stored at listing *(continued)*

```
1  ```move
2  // deposit_vault.move  calc_and_validate_deposit
3  let state = borrow_state_mut();
4  vault_base::assert_token_exists(&state.base, token_in);
5  let token_decimals = vault_base::token_decimals(&state.base, token_in);
```

Impact

Low. If a listed token's issuer lowers its decimals, the same native balance is read as a much larger base-8 amount, depositing 1,000 USDCx at 6 decimals mints 1,000 zOPAL, and at 4 decimals mints 100,000 zOPAL. Raising decimals above 8 makes $\text{pow}10((8 - \text{token_decimals}) \text{ as } u64)$ underflow and permanently aborts every deposit and non-fiat redemption for that token. This requires an external issuer to act, and the tokens listed are chosen by the trusted admin.

Likelihood

Low. Requires a third-party issuer to mutate metadata after listing. The dependency is real rather than hypothetical, `deploy.config.json` points at an externally supplied USDCx metadata address.

Recommendation

Record the decimals at listing time and read them from config everywhere else. After the change, `fungible_asset::decimals` should appear exactly once per vault, in `add_payment_token`. That is worth enforcing with a CI grep.

Low Severity Issues

initialize skips the sanctions-list validation that set_sanctions_list performs

Resolved

Path

```
move/zvault_core/sources/deposit_vault.move,  
move/zvault_core/sources/redemption_vault.move,  
move/zvault_core/sources/vault_base.move
```

Function Name

```
initialize, set_sanctions_list, new_vault_state
```

Description

The setter validates the address; the constructor does not. `new_vault_state` does not validate it either. The doc-comment on the setter spells out exactly why it matters: storing an address with no sanctions list on it makes every `is_sanctioned` read abort with `E_NO_SANCTIONS_LIST`, which blocks all deposits and redemptions. The guard is present on the path that can be retried and missing from the one that runs only once.

```
1 // deposit_vault.move set_sanctions_list validates  
2 public entry fun set_sanctions_list(admin: &signer, sanctions_list: address) acquires DepositVaultState {  
3     assert_vault_admin(admin);  
4     // Reject a contractless list address: storing one would brick every  
5     // deposit/redemption via is_sanctioned (fail-safe at config time).  
6     compliance::assert_sanctions_list_valid(sanctions_list);  
7     vault_base::set_sanctions_list(&mut borrow_state_mut().base, signer::address_of(admin), sanctions_list);  
8 }
```

initialize skips the sanctions-list validation that set_sanctions_list performs *(continued)*

```
1 // deposit_vault.move initialize passes the same address straight through, unchecked
2 let base = vault_base::new_vault_state(
3     vault_base::deposit_vault_identity(),
4     tokens_receiver,
5     fee_receiver,
6     instant_fee_bps,
7     instant_daily_limit,
8     variation_tolerance_bps,
9     min_amount,
10    sanctions_list,
11 );
```

```
1 // deposit_vault.move and redemption_vault.move initialize
2 assert!(signer::address_of(publisher) == @zvault_core, E_NOT_PUBLISHER);
3 assert!(!exists<DepositVaultState>(@zvault_core), E_ALREADY_INITIALIZED);
4 compliance::assert_sanctions_list_valid(sanctions_list);
5 let base = vault_base::new_vault_state(...);
```

Impact

Low. A mis-ordered deploy pointing at an address that has not yet run `create_sanctions_list` aborts `deposit_instant`, `deposit_request`, `redeem_instant`, `redeem_request` and `redeem_fiat_request` from genesis. It is recoverable with `set_sanctions_list`, and no funds are at risk.

initialize skips the sanctions-list validation that set_sanctions_list performs *(continued)*

Likelihood

Low. Requires the publisher key and a specific deploy-ordering mistake. It is latent today only because `deploy.config.json` ships `"sanctions_list": "0x0"` and that file's own comment says production swaps the addresses.

Recommendation

Validate in the constructor as well. More generally: for every setter that validates a parameter, the matching initialize parameter should carry the same validation.

Low Severity Issues

The settlement store has no obligation tracking, and request_redeemer is never enforced

Resolved

Path

`move/zvault_core/sources/redemption_vault.move`

Function Name

`withdraw_settlement, fund_settlement, set_request_redeemer`

Description

The zOPAL escrow has a tracked obligation and a withdrawal guard. The settlement store, which funds every non-flat approval, has neither a counter nor a guard, and the destination is fully caller-supplied.

Separately, request_redeemer is recorded, settable, documented as "the canonical settlement-withdrawal recipient", and never read by any fund-moving function. The setter enforces a non-zero address, so the value is validated on write and unused on read.

Note that this does not touch escrowed zOPAL, REDEMPTION_VAULT_SEED and REDEMPTION_SETTLEMENT_SEED are different objects, and withdraw_settlement only reaches the latter.

```
1 // redemption_vault.move withdraw_token protects pending-request backing
2 if (token == zopal_token::metadata_address()) {
3     let bal = primary_fungible_store::balance(
4         vault_base::redemption_vault_identity(), zopal_token::metadata(),
5     );
6     assert!(bal >= amount + state.total_escrowed_z_token, E_ESCROW_OBLIGATION);
7 };
```

The settlement store has no obligation tracking, and request_redeemer is never enforced *(continued)*

```
1 // redemption_vault.move withdraw_settlement
2 public entry fun withdraw_settlement(
3     admin: &signer,
4     token: address,
5     amount: u64,
6     withdraw_to: address,
7 ) acquires RedemptionVaultState, Identity {
8     assert_vault_admin(admin);
9     let _ = borrow_state();
10    let identity = borrow_global<Identity>(@zvault_core);
11    let settlement_signer =
12        object::generate_signer_for_extending(&identity.settlement_extend_ref);
13    let fa = primary_fungible_store::withdraw(&settlement_signer, token_metadata(token), amount);
14    primary_fungible_store::deposit(withdraw_to, fa);
15    ...
16 }
```

```
1 // redemption_vault.move withdraw_settlement
2 public entry fun withdraw_settlement(
3     admin: &signer,
4     token: address,
5     amount: u64,
6     withdraw_to: address,
7 ) acquires RedemptionVaultState, Identity {
8     assert_vault_admin(admin);
9     let state = borrow_state();
10    assert!(withdraw_to == state.request_redeemer, E_NOT_REQUEST_REDEEMER);
11    let owed = *smart_table::borrow_with_default(&state.total_pending_settlement, token, &0u64);
12    let bal = primary_fungible_store::balance(
13        vault_base::redemption_settlement_identity(), token_metadata(token),
14    );
15    assert!(bal >= amount + owed, E_SETTLEMENT_OBLIGATION);
16    ...
17 }
```

The settlement store has no obligation tracking, and request_redeemer is never enforced *(continued)*

Impact

Low. A vault admin can empty the settlement store with no abort, after which every pending approval aborts with E_INSUFFICIENT_SETTLEMENT while requesters' zOPAL stays escrowed. Recovery exists through reject_request plus withdraw_token, but it is manual and one request at a time. The request_redeemer gap is a documentation-versus-code mismatch that misleads anyone relying on the documented constraint.

Likelihood

Low. Only reachable by the trusted vault-admin role acting mistakenly. No unprivileged path gets here.

Recommendation

Track the pending settlement obligation per token and guard withdrawals against it, mirroring withdraw_token. total_pending_settlement is incremented in do_redeem_request and decremented in approve_request_internal and reject_request. If enforcing request_redeemer as the destination is not wanted, then update the documentation to describe the field as informational only but do not leave both in place.

Low Severity Issues

Economic setters accept any value with no bounds or coherence checks

Resolved

Path

```
move/zvault_core/sources/vault_base.move,  
move/zvault_core/sources/deposit_vault.move
```

Function Name

```
set_min_amount, set_min_first_deposit, set_max_supply_cap
```

Description

set_min_amount performs no validation at all, it assigns and emits. set_max_supply_cap is the same shape. There is no floor, no ceiling and no check against current protocol state. max_supply_cap = 0, or any value at or below the current total minted, blocks all further minting on both deposit paths. min_first_deposit = U64_MAX blocks every first deposit. min_amount = 0 silently removes the general floor.

```
1 // vault_base.move set_min_amount  
2 public(friend) fun set_min_amount(state: &mut VaultState, caller: address, min_amount: u64) {  
3     state.min_amount = min_amount;  
4     event::emit(SetMinAmount { vault: state.vault_identity, caller, min_amount });  
5 }
```

```
1 // deposit_vault.move set_min_first_deposit: role check only  
2 public entry fun set_min_first_deposit(admin: &signer, value: u64) acquires DepositVaultState {  
3     assert_vault_admin(admin);  
4     borrow_state_mut().min_first_deposit = value;  
5     event::emit(SetMinFirstDeposit { caller: signer::address_of(admin), value });  
6 }
```

Economic setters accept any value with no bounds or coherence checks *(continued)*

```
1 // vault_base.move set_min_amount
2 public(friend) fun set_min_amount(state: &mut VaultState, caller: address, min_amount: u64) {
3     assert!(min_amount > 0, E_INVALID_MIN_AMOUNT);
4     state.min_amount = min_amount;
5     event::emit(SetMinAmount { vault: state.vault_identity, caller, min_amount });
6 }
```

```
1 // deposit_vault.move set_min_first_deposit
2 public entry fun set_min_first_deposit(admin: &signer, value: u64) acquires DepositVaultState {
3     assert_vault_admin(admin);
4     let state = borrow_state_mut();
5     assert!(value >= vault_base::min_amount(&state.base), E_INVALID_MIN_FIRST_DEPOSIT);
6     state.min_first_deposit = value;
7     event::emit(SetMinFirstDeposit { caller: signer::address_of(admin), value });
8 }
```

```
1 // deposit_vault.move set_max_supply_cap
2 public entry fun set_max_supply_cap(admin: &signer, value: u128) acquires DepositVaultState {
3     assert_vault_admin(admin);
4     assert!(value >= zopal_token::total_supply(), E_CAP_BELOW_SUPPLY);
5     borrow_state_mut().max_supply_cap = value;
6     event::emit(SetMaxSupplyCap { caller: signer::address_of(admin), value });
7 }
```

Economic setters accept any value with no bounds or coherence checks *(continued)*

Impact

Low. A misconfiguration passes silently and denies the core deposit function to all users until it is corrected. It is fixable with a routine role-gated transaction, so no funds are at risk unless it is combined with a lost DEFAULT_ADMIN_ROLE, in which case there is no one left to correct it.

Likelihood

Low. Requires the trusted vault admin to pass a wrong value.

Recommendation

Add sanity bounds and a cross-parameter coherence check to all three setters.

Low Severity Issues

Removing a payment token strands every pending request denominated in it

Resolved

Path

```
move/zvault_core/sources/vault_base.move,  
move/zvault_core/sources/redemption_vault.move
```

Function Name

```
remove_payment_token, require_and_update_allowance, approve_request_internal
```

Description

`remove_payment_token` deletes the token from both the list and the config table, with no check for outstanding requests. Approval then calls `require_and_update_allowance` for the request's payout token, which takes its "token not configured" branch and aborts with `E_EXCEED_ALLOWANCE`. Every pending request in that token becomes permanently unapprovable while the requester's zOPAL stays escrowed. `request.token_out_rate` is also a cached divisor from the earlier transaction with no fallback.

```
1 // vault_base.move remove_payment_token  
2 public(friend) fun remove_payment_token(state: &mut VaultState, caller: address, token: address) {  
3     let (found, idx) = vector::index_of(&state.payment_tokens, &token);  
4     assert!(found, E_TOKEN_NOT_EXISTS);  
5     vector::remove(&mut state.payment_tokens, idx);  
6     smart_table::remove(&mut state.token_configs, token);  
7     event::emit(RemovePaymentToken { vault: state.vault_identity, caller, token });  
8 }
```

Removing a payment token strands every pending request denominated in it *(continued)*

```
1 // redemption_vault.move remove_payment_token
2 public entry fun remove_payment_token(admin: &signer, token: address) acquires RedemptionVaultState {
3     assert_vault_admin(admin);
4     let state = borrow_state_mut();
5     let pending = *smart_table::borrow_with_default(&state.pending_by_token, token, &0u64);
6     assert!(pending == 0, E_TOKEN_HAS_PENDING_REQUESTS);
7     vault_base::remove_payment_token(&mut state.base, signer::address_of(admin), token);
8 }
```

Impact

Low. Affected redemptions cannot settle. The escrowed zOPAL is recoverable through `reject_request` followed by `withdraw_token`, but only one request at a time, with no batch form.

Likelihood

Low. Requires an admin to de-list a token while requests are in flight. That is plausible as part of a depeg response, which is exactly when the request queue is most likely to be full.

Recommendation

Refuse to remove a token that still has pending requests. `pending_by_token` is incremented in `do_redeem_request` and decremented in `approve_request_internal` and `reject_request`. If a hard block is too rigid for an emergency de-listing, add a batch reject-and-refund helper and document a drain-the-queue step in the token-removal runbook.

Low Severity Issues

A rejected deposit request never returns the token allowance it consumed

Acknowledged

Path

```
move/zvault_core/sources/deposit_vault.move,  
move/zvault_core/sources/vault_base.move
```

Function Name

```
calc_and_validate_deposit, reject_request, require_and_update_allowance
```

Description

The per-token allowance is consumed on the full input amount when the request is created. `reject_request` only changes the status. The allowance is not restored here or anywhere else. Only `change_token_allowance` can top it back up.

```
1 // deposit_vault.move  calc_and_validate_deposit  
2 // Consume the per-token allowance on the full input amount (EVM order:  
3 // before fee computation).  
4 vault_base::require_and_update_allowance(&mut state.base, token_in, amount8);
```

A rejected deposit request never returns the token allowance it consumed *(continued)*

```
1 // deposit_vault.move reject_request
2 public entry fun reject_request(admin: &signer, request_id: u64) acquires DepositVaultState {
3     assert_vault_admin(admin);
4     let state = borrow_state_mut();
5     assert!(smart_table::contains(&state.requests, request_id), E_REQUEST_NOT_EXIST);
6     let request = smart_table::borrow_mut(&mut state.requests, request_id);
7     assert!(request.status == STATUS_PENDING, E_REQUEST_NOT_PENDING);
8     request.status = STATUS_CANCELED;
9     event::emit(RejectRequest { request_id, sender: request.sender });
10 }
```

```
1 // deposit_vault.move DepositRequest gains the consumed amount
2 struct DepositRequest has store, drop, copy {
3     sender: address,
4     token_in: address,
5     status: u8,
6     deposited_usd_amount: u128,
7     usd_amount_without_fees: u128,
8     token_out_rate: u128,
9     /// Base-8 allowance consumed at creation; restored if the request is rejected.
10    allowance_consumed: u128,
11 }
```

A rejected deposit request never returns the token allowance it consumed *(continued)*

```
1 // deposit_vault.move reject_request restores the allowance
2 public entry fun reject_request(admin: &signer, request_id: u64) acquires DepositVaultState {
3     assert_vault_admin(admin);
4     let state = borrow_state_mut();
5     assert!(smart_table::contains(&state.requests, request_id), E_REQUEST_NOT_EXIST);
6     let request = *smart_table::borrow(&state.requests, request_id);
7     assert!(request.status == STATUS_PENDING, E_REQUEST_NOT_PENDING);
8
9     vault_base::restore_allowance(&mut state.base, request.token_in, request.allowance_consumed);
10
11     let stored = smart_table::borrow_mut(&mut state.requests, request_id);
12     stored.status = STATUS_CANCELED;
13     event::emit(RemoveRequest { request_id, sender: request.sender });
14 }
```

```
1 // vault_base.move restore_allowance, skipping the unlimited sentinel
2 public(friend) fun restore_allowance(state: &mut VaultState, token: address, amount8: u128) {
3     if (!smart_table::contains(&state.token_configs, token)) { return };
4     let config = smart_table::borrow_mut(&mut state.token_configs, token);
5     if (config.allowance == MAX_ALLOWANCE) { return };
6     config.allowance = config.allowance + amount8;
7 }
```

Impact

Low. A token's lifetime deposit capacity is burned down by requests that were never settled. Once it reaches zero, legitimate deposits in that token abort until an admin manually raises the allowance. No funds are lost.

Likelihood

Medium. Any user can submit requests, and rejection is the normal operational response to a bad one, so the leak accrues through ordinary use rather than needing deliberate abuse.

A rejected deposit request never returns the token allowance it consumed *(continued)*

Recommendation

Store the consumed amount on the request and give it back on rejection. The alternative is to consume the allowance at approval rather than at creation, so only settled volume counts against it.

Informational Severity Issues

zopal_token stores a TransferRef and an ExtendRef that no code ever uses

Resolved

Path

move/zvault_core/sources/zopal_token.move, move/zvault_core/Move.toml

Function Name

init_module, Refs

Description

init_module generates four capability refs and stores all of them, but only two are ever read. mint_to uses mint_ref and burn_from uses burn_ref. Nothing anywhere reads transfer_ref or extend_ref, and there is no path that burns or destroys either one.

A TransferRef on FA metadata grants freeze and unfreeze on arbitrary stores plus transfers that ignore frozen state. The ExtendRef yields the metadata object's signer.

zopal_token stores a **TransferRef** and an **ExtendRef** that no code ever uses *(continued)*



```
1  ``move
2  // zopal_token.move
3  struct Refs has key {
4      mint_ref: MintRef,
5      burn_ref: BurnRef,
6      transfer_ref: TransferRef,
7      extend_ref: ExtendRef,
8  }
```

zopal_token stores a TransferRef and an ExtendRef that no code ever uses *(continued)*

```
1 // zopal_token.move
2 struct Refs has key {
3     mint_ref: MintRef,
4     burn_ref: BurnRef,
5 }
```

Impact

Informational. Nothing today can reach either ref. The point is that zvault_core/Move.toml carries no upgrade_policy, so it publishes as compat, a later compatible upgrade could start using them and gain freeze and seize powers over zOPAL with no governance step and no on-chain notice, which contradicts the module's documented "plain FA, no transfer restrictions" posture.

Likelihood

Low. This is latent authority, not a live exploit path.

zopal_token stores a TransferRef and an ExtendRef that no code ever uses *(continued)***Recommendation**

Either drop the refs if they are vestigial, with init_module no longer generating transfer_ref or extend_ref, or keep them and document them explicitly as a reserved future compliance lever, together with the governance process that would have to approve their use and the upgrade policy that constrains it.

Informational Severity Issues

friend declarations grant more access than any caller actually uses

Acknowledged

Path

```
move/zvault_core/sources/vault_base.move,  
move/zvault_core/sources/zopal_token.move
```

Function Name

```
module-level friend declarations
```

Description

vault_base declares friend zvault_core::zopal_token, but zopal_token only calls deposit_vault_identity() and redemption_vault_identity(), both #[view] public fun, which need no friendship at all. The declaration hands over the entire public(friend) surface: new_vault_state, pause, unpause, set_greenlist_enable, set_sanctions_list, require_and_update_allowance and the rest.

The same is true in reverse. zopal_token friends both vaults, so redemption_vault can reach mint_to and deposit_vault can reach burn_from, even though neither does.



```
1 // vault_base.move remove this line  
2 friend zvault_core::zopal_token;
```

Impact

Informational. Inert today, the vault_base friend functions all need a &mut VaultState, which zopal_token cannot obtain. The issue is that the trust boundary is drawn wider than anything actually uses.

friend declarations grant more access than any caller actually uses*(continued)***Likelihood**

Low. No current path exercises the extra surface. The risk is a future edit that does so without anyone noticing the boundary was already open.

Recommendation

Narrow the declarations to match real usage. Drop friend `zvault_core::zopal_token` from `vault_base` entirely, since the two functions `zopal_token` needs are already public views. In `zopal_token`, keep only the edges that are used, and if the module ever needs to distinguish which vault may mint versus burn, split the friend surface rather than granting both to both.

Functional Tests

ZVault

- ✓ Instant deposit mints the correct zOPAL amount at the current NAV, to the caller or to a custom recipient, and splits the payment token between the tokens receiver and the fee receiver
- ✓ Deposit request takes the funds immediately, mints nothing, and mints at the admin-supplied rate only when the request is approved
- ✓ Rejecting a pending deposit request cancels it cleanly and mints nothing
- ✓ Instant redemption burns the net zOPAL from the caller and pays the correct payout amount out of the instant buffer
- ✓ Redemption request escrows the net zOPAL in the vault and pays the redeemer from the settlement store when approved
- ✓ Fiat redemption request records the manual-fulfilment sentinel, applies both the flat and additional fiat fees, moves no tokens on-chain, and settles only after the fiat allowance step
- ✓ Rejecting a pending redemption request releases the escrow obligation and the escrowed zOPAL is refundable to the user
- ✓ Escrow and store accounting holds: the escrow counter tracks requests, `withdraw_token` cannot draw below the outstanding escrow, and the instant buffer and settlement store stay separate
- ✓ Fees are computed on the correct basis, truncated to the payment token's native precision, and waived-fee accounts pay zero
- ✓ Decimal conversion and rounding always favour the protocol across 2, 4, 6 and 8 decimal tokens, and a deposit-then-redeem cycle never returns more than was put in
- ✓ Slippage bounds are enforced on both instant paths and pass at the exact boundary
- ✓ Maximum supply cap blocks minting on both the instant and the approval path, inclusive at the boundary
- ✓ `min_amount` and `min_first_deposit` are enforced against the post-fee amount, and `free_from_min` accounts bypass both
- ✓ Daily volume limit is enforced, inclusive at the boundary, and resets at the UTC day boundary

- ✓ Per-token allowance is consumed on use and blocks the operation once exhausted, while the unlimited sentinel is never decremented
- ✓ Compliance gates block blacklisted, sanctioned and non-greenlisted parties on every user-facing entry point, including custom recipients
- ✓ Fiat redemption always requires greenlist membership regardless of the greenlist toggle
- ✓ pause and pause_fn block the intended entry points, unpause restores them, and the two vaults pause independently
- ✓ Access control holds: only the vault admin can approve requests or change configuration, and zOPAL mint and burn are reachable only from their respective vaults
- ✓ A stale or never-set NAV blocks every value-bearing path in both vaults

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of Zoth. We performed our audit according to the procedure described above.

16 issues in ZVault were found which were resolved by the Zoth team.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

September 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com